



ADR-006: Field-level encryption (AES-256-GCM, key ids) + HMAC blind index

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-006	1.0	07 October 2026	iorta TechNXT Architecture	Accepted

- **Related:** [ADR-004](#), [Security Architecture](#), section 9, [Data Architecture](#)

Context and problem statement

The platform holds personal data on millions of Vietnamese vehicle owners: names, phone numbers, call transcripts, claim descriptions and locations, and staff TOTP secrets. Vietnam's personal data protection regime requires appropriate technical protection of personal data: Decree 13/2023/ND-CP and the Law on Personal Data Protection (Law No. 91/2025/QH15, effective 01 January 2026) (**confirm with TASCO legal**). Disk and TDE encryption alone do not protect against:

- a database operator or DBA reading data;
- leaked backups or replicas;
- SQL-level exfiltration through a compromised read path.

Equality lookup by phone is still needed, for example to match an inbound call or a duplicate-detection key.

Decision drivers

- PII must be unreadable at rest without the application key, including in backups, replicas and BI extracts.
- Keys must be rotatable without downtime.
- Equality search on selected PII must be possible without decryption.
- No third-party crypto library ([ADR-002](#)).

Considered options

1. Application-level AES-256-GCM per field with key ids in the ciphertext, plus HMAC-SHA256 blind index columns.
2. Storage/TDE encryption only (managed disk encryption).
3. `pgcrypto` in the database.
4. A cloud KMS envelope per record (encrypt via a KMS API call).

Decision outcome

Chosen option: **1**, with infrastructure encryption at rest **in addition** (managed Postgres storage encryption and TLS in transit).

- **Format** (`src/shared/crypto.js#createFieldCipher`): `enc:v1:<keyId>:<base64(iv[12] || tag[16] || ciphertext)>`. The IV is random per encryption. The plaintext is `JSON.stringify(value)`, so arrays and objects work, for example `altPhones` and `transcript`.
- **Keys**.
 - `DATA_KEYS="k1:<b64 32B>,k2:<b64 32B>"` (or `DATA_KEYS_FILE`). The active key is `DATA_KEY_ACTIVE`, defaulting to the last listed key.
 - Production refuses to start without keys (`src/shared/config.js`). Each key must decode to exactly 32 bytes.
 - Decryption selects the key by the id embedded in the value.

- **Rotation.** Add `k2` to `DATA_KEYS` and set `DATA_KEY_ACTIVE=k2`. New writes use `k2` and old values still decrypt with `k1`. Records re-encrypt lazily on their next write. Once a **re-key job** has rewritten all rows, remove `k1`. The re-key job is **planned**: iterate each collection with keyset paging and `update()` each document. `cipher.keyIdOf()` already exists for progress reporting.
- **Fields encrypted** (`schema.js` `pii`):
 - `source_records.{phoneRaw, fullName}`;
 - `profiles.{name, phone, altPhones}`;
 - `messages.to`;
 - `voice_sessions.transcript`;
 - `handoffs.name`;
 - `claims.{description, location}`;
 - `users.{totpSecret, displayName}`.
- **Blind index.** `profiles.phone_bidx = HMAC-SHA256(BLIND_INDEX_KEY, lower(phone))`. It is a separate key, so it does not encrypt and is not derived from a data key. It is computed in `codec.encode`.
- **Indexed columns** are never PII by convention. They hold region, dates, status, scores and the plate key (see the note below).

Consequences

- Good: DB dumps, replicas and logs contain ciphertext for PII. The JSON logger additionally redacts PII keys (`src/shared/logger.js`).
- Good: rotation does not stop the platform.
- Bad: no range, prefix or `LIKE` search on encrypted fields. This is acceptable because search is by plate.
- Bad: the blind index is deterministic. Equal phone numbers are linkable inside the database, and the small Vietnamese mobile number space (about 10^9) is brute-forceable if `BLIND_INDEX_KEY` leaks. Keep that key in the secret store with the same protection as data keys. Rotating it requires recomputing every `*_bidx` column (planned job). Today `phone_bidx` is written but **not yet queried** by any use case.
- Bad: the keys live in application memory. A KMS envelope (DEKs wrapped by a KMS KEK, unwrapped at boot) is the **target** in production ([Deployment and Infrastructure Architecture](#), section 8).
- **Note — licence plate.** The plate is the MDM match key and the profile `id`, so it is stored in clear: in `profiles.id`, `policies.plate`, `quotes.plate`, `handoffs.plate`, message texts and event payloads. A plate can identify a natural person indirectly. **Confirm with TASCO legal / DPO** whether plates are treated as personal data. If so, the target is to use a keyed pseudonym (`HMAC(plate)`) as the `id`, and to keep the display plate encrypted.

Pros and cons of the options

TDE only. Transparent. However, it does nothing against DBA access or logical dumps.

`pgcrypto`. Keys travel to the DB in SQL statements (visible in logs and `pg_stat_statements`), and it couples the code to Postgres.

KMS per record. The strongest key custody. However, it adds latency and cost per field at millions of records. Use KMS for the KEK only.